Logbook Entry Generator

Developer Guide

PionCT Experiment — Jefferson Lab

June 24, 2026

Contents

1 Overview **2**

2 `build.py` Architecture **3**
- 2.1 Entry Point . . . 3
- 2.2 Field Partitioning . . . 3
- 2.3 CSS Class to JS Variable Name . . . 4
- 2.4 Schema Hash . . . 4
- 2.5 Python Version Compatibility . . . 4

3 Generated HTML Tool Architecture **5**
- 3.1 State Management . . . 5
- 3.2 Run Entry DOM Lifecycle . . . 5
- 3.3 Google Sheets Export Flow . . . 5
- 3.4 Import from HTML . . . 6

4 Adding a New Field **7**

5 Adding a New Field `source` Type **8**

6 Adding a New Field `type` **9**

7 Modifying the CSS / Visual Theme **10**

8 Deployment Checklist **11**

9 Known Limitations and Technical Debt **12**

1. Overview

This document covers the internal architecture of the build system and generated HTML tool, intended for anyone modifying or extending the code.

The system has two layers:

1. **`build.py`** — a Python 3.6+ script that reads `config.json` and generates `output/logbook_generator.html` and (optionally) `output/apps_script.gs`.
2. **`output/logbook_generator.html`** — a self-contained single-page application that runs entirely in the browser. No framework, no bundler, no server.

The HTML tool is not hand-edited after generation. All customisation happens in `config.json`; re-running `build.py` regenerates the outputs.

2. build.py Architecture

The script is structured as a pipeline of pure functions, each responsible for generating one fragment of the final HTML or Apps Script.

2.1 Entry Point

```
main()
  +-- load_config()           validate config.json
  +-- compute_schema_hash()   detect Sheets-breaking field changes
  +-- embed_logo()            base64-encode logo or fall back to
      path
  +-- build_html()            assemble the full HTML string
  |     +-- build_add_run_entry_fields()       <-- form inputs
  |     +-- build_generate_html_run_table()    <-- generateHTML() JS
  |     +-- build_extract_run_data()           <-- extractRunData()
      JS
  |     +-- build_save_run_entries()           <--
      saveToLocalStorage() JS
  |     +-- build_load_run_entries()           <--
      loadFromLocalStorage() JS
  |     +-- build_import_run_entries()         <-- importFromHTML()
      JS
  |     +-- build_quick_links_form()           <-- form HTML
  |     +-- build_quick_links_save_js()        <-- save JS fragment
  |     +-- build_quick_links_load_js()        <-- load JS fragment
  |     +-- build_quick_links_clear_js()       <-- import clear JS
      fragment
  |     +-- build_quick_links_generate_html() <-- generateHTML()
      JS fragment
  |     `-- build_import_links()              <-- importFromHTML()
      link matcher
  `-- build_apps_script()     assemble the Apps Script string
```

All `build_*` functions take the parsed config (or a subset of it) and return a string. They have no side effects — they do not write files or access the DOM.

2.2 Field Partitioning

A core concept used throughout the script is splitting `run_fields` into subsets:

```
user_fields       = [f for f in fields if f["source"] == "user"]
auto_form_fields  = [f for f in fields if f["source"] == "auto"
                                       and f["key"] != "date"]
form_fields       = [f for f in fields if f["source"] in
    ("user","auto")
                                       and f["key"] != "date"]
html_table_fields = [f for f in fields if f["source"] !=
    "sheet-only"]
```

Subset	Used for
`form_fields`	Inputs rendered in the `addRunEntry()` template; save/load loops
`html_table_fields`	Column headers and data cells in `generateHTML()` and `importFromHTML()`
All fields	`apps_script.gs` HEADERS array and row construction

`date` is excluded from `form_fields` because it is auto-generated at send/generate time and never shown as an input; it is included in `html_table_fields` so it appears in the generated logbook HTML table.

2.3 CSS Class to JS Variable Name

`js_key(css_class)` converts a CSS class to a JS variable name:

```
run-hms-p       ->  strip "run-"  ->  hms-p  ->  camelCase  ->  hmsP
run-comments    ->  strip "run-"  ->  comments               ->
    comments
date            ->  no prefix     ->  date                   ->
    date
```

This name is used consistently across `extractRunData`, `saveToLocalStorage`, `loadFromLocalStorage`, and the Apps Script row array.

2.4 Schema Hash

`compute_schema_hash()` hashes the ordered list of `(key, sheet_column)` tuples for all fields. This hash is stored in `.schema_hash` after each build. On the next build, if the hash has changed and Sheets is enabled, a warning is printed.

The hash is also embedded as a comment in the generated HTML and Apps Script so the deployed version can be identified.

> If `.schema_hash` is missing (deleted or first build), the comparison is skipped silently and a new file is written. No data is lost.

2.5 Python Version Compatibility

The script requires **Python 3.6+**. It uses only the standard library: `base64`, `hashlib`, `json`, `mimetypes`, `os`, `sys`, `pathlib`.

No `removeprefix` (3.9+), no walrus operator (3.8+), no match statements (3.10+).

3. Generated HTML Tool Architecture

3.1 State Management

All mutable state lives in one place: the browser's `localStorage`, under the key `{experiment_name_lowercase}Lo` The key is derived at build time from `experiment_name` in the config.

```
localStorage["pionctLogbookData"] = {
  summaryText, runPlanLink, checklistLink, ...   (fixed fields)
  sheetsWebAppUrl, sheetsSharedSecret,           (if Sheets
     enabled)
  additionalLinks: [ {text, url}, ... ],
  shiftEntries:    [ {time, entry}, ... ],
  runEntries:      [ {runNumber, startTime, ..., sentToSheet}, ...
     ],
  timestamp: "ISO 8601 string"
}
```

`saveToLocalStorage()` is called after every user interaction that changes data. `loadFromLocalStorage()` is called on `DOMContentLoaded`.

3.2 Run Entry DOM Lifecycle

`addRunEntry()` creates a new `.entry-item` div and appends it to `#runEntries`. All data for a run lives in the DOM of its `.entry-item` — there is no separate JS data structure. Save/load/export all operate by querying the DOM:

```
document.querySelectorAll('#runEntries .entry-item').forEach(item
   => {
    const value = item.querySelector('.run-hms-p').value;
    ...
});
```

This means the DOM is the single source of truth during a session. The `sentToSheet` state is stored as a `data-sent` attribute on the `.sheet-led` span, read back by `saveToLocalStorage()`.

3.3 Google Sheets Export Flow

```
sendRunToSheet(button)
  '-- extractRunData(item)          builds the JSON payload
  '-- postRunToSheet(runData)
        '-- fetch(webAppUrl, POST) sends JSON body
        '-- Apps Script doPost()  appends row to sheet
        '-- returns {ok, rowAppended}
  '-- updates LED + status text
  '-- saveToLocalStorage()          persists the green LED state
```

The `Content-Type` is set to `text/plain;charset=utf-8` intentionally. Apps Script Web Apps do not respond correctly to CORS preflight requests, which `application/json` would trigger. Using `text/plain` sends the request as a "simple request" (no preflight) while the Apps Script reads the raw body via `e.postData.contents` regardless of declared content type.

3.4 Import from HTML

`importFromHTML()` uses `DOMParser` to parse the previously-generated logbook HTML, then identifies tables by their header text:

- A table with headers `Time` and `Logbook Entry` → shift log table
- A table with headers `Run Number` and `Start Time` → run summary table

Fields are restored by column index. If the field list has changed between the session that generated the HTML and the importing tool, columns may misalign silently. This is a known limitation (see Chapter 9).

4. Adding a New Field

1. Add an entry to `run_fields` in `config.json`:

```
{
  "key":          "run-my-field",
  "label":        "My Field",
  "type":         "text",
  "source":       "user",
  "sheet_column": "My Field",
  "sheet_align":  "left"
}
```

2. Run `python3 build.py`.
3. The field appears automatically in the form, the HTML table, save/load, extract, and (if Sheets enabled) the Apps Script row.
4. If Sheets is enabled and the schema hash changed, redeploy the Apps Script.

No changes to `build.py` or the HTML template are needed for a standard `text` or `textarea` field with `source` of `user`, `auto`, or `sheet-only`.

5. Adding a New Field `source` Type

Currently supported: `user`, `auto`, `sheet-only`.

To add a new type (e.g. `computed` with a formula, or `dropdown`):

1. Add handling in `build_add_run_entry_fields()` — the function that generates the form input HTML for each field.
2. Update `VALID_SOURCES` in `load_config()` to accept the new value.
3. Decide which partition sets the new source belongs to (form, table, both) and update the partition logic at the top of `build_html()`.
4. If the new type changes what gets sent to the sheet, update `build_apps_script()` accordingly.

6. Adding a New Field type

Currently supported: `text` (single-line input), `textarea` (multi-line).

To add `select` (dropdown):

1. Add `"select"` to `VALID_TYPES` in `load_config()`.
2. In `build_add_run_entry_fields()`, add a branch that emits a `<select>` element. You will also need a new config key (e.g. `"options": [...]`) to carry the dropdown values.
3. The save/load/export machinery reads `.value` from the DOM element and does not need changes — `<select>` elements expose `.value` the same as inputs.

7. Modifying the CSS / Visual Theme

The entire CSS is a single `<style>` block in the generated HTML, generated verbatim by `build_html()` in `build.py`. The colour constants are:

Role	Value
Accent / headings	`#ff9800` (orange)
Secondary accent	`#ffb74d` (light orange)
Container background	`linear-gradient(135deg, #6b7176, #5a6066)`
Page background	`#e6e6e6`
Input background	`#464B50`
Body text (on page bg)	`#2C3539`
Label text (on container)	`#ffffff`

To change the theme, modify the CSS string inside `build_html()` in `build.py` and rebuild. Do not edit the generated HTML directly — it will be overwritten on the next build.

8. Deployment Checklist

When releasing a new version:

- Edit `config.json` as needed
- Run `python3 build.py`
- Check for the schema hash change warning
- If hash changed and Sheets is enabled: paste new `apps_script.gs`, create a new deployment, update the Web App URL in the tool
- Test the generated HTML in a browser (add a run entry, send to sheet, reload and verify session restores, test Import from Previous HTML)
- Commit `config.json`, `build.py`, and `docs/` to the repository
- Do **not** commit `output/` (generated files) or `.schema_hash` unless your team specifically wants to track generated artefacts

9. Known Limitations and Technical Debt

- `importFromHTML` **link matching** uses keyword heuristics on link text (first word of each quick-link label). If two links share a first word, one will shadow the other. A future fix would be to embed machine-readable metadata in the generated HTML.
- **No field validation** — all inputs are `type="text"`. Numeric fields (beam current, momentum, etc.) accept arbitrary strings. Downstream errors in the sheet or generated HTML are possible if non-numeric values are entered.
- **Single Apps Script deployment URL** is stored in localStorage. If a second coordinator updates the deployment and the URL changes, each browser running the tool needs the new URL updated manually.
- `totalTime` **computation** treats any invalid `HH:MM` string as "leave unchanged" rather than clearing the field. A manually entered value survives a start/stop edit only if both times become invalid simultaneously.